

Ocean University of China
Robotics Integration Group Project
Laboratory Report

**Desktop Object Detection and Real-Time
Recognition with YOLO11n**

Student: Chengyang Deng
Student ID: 24020036012
Class: Class 1, Grade 2024
Course: Robotics Integration Group Project
Instructor: Xiaowei Zhou
University: Ocean University of China

September 2026

Abstract

This experiment develops a four-class desktop-object detector for mouse, keyboard, laptop, and headphones using the lightweight YOLO11n architecture. The final dataset contains 1,343 images and 1,878 horizontal bounding boxes. It combines 147 captured or user-provided images, 598 controlled scene-expanded images, and 598 images derived from Open Images V7. The data are organized into 1,047 training images, 173 validation images, and 123 independent test images. A pretrained YOLO11n checkpoint is fine-tuned at an input size of 640 pixels. The recorded run contains 89 complete epochs, and the checkpoint selected at epoch 75 achieves validation precision 0.848, recall 0.855, mAP@0.5 0.888, and mAP@0.5:0.95 0.661. On the independent test split, the selected model achieves overall precision 0.882, recall 0.762, mAP@0.5 0.810, and mAP@0.5:0.95 0.564. Under a fixed confidence threshold of 0.25 and an IoU matching threshold of 0.5, 174 of 213 annotated test objects are matched correctly, corresponding to a recognition rate of 81.69 percent. The implementation stores annotated predictions and representative error cases, and it runs real-time USB-camera inference on the development computer while displaying class names, bounding boxes, confidence scores, and frame rate. A provided 28.2-second screen recording preserves the camera demonstration; only the camera-view region is reproduced in this report. The selected PyTorch checkpoint is also exported to a fixed-shape ONNX model. Dedicated programs now implement repeatable Jetson camera benchmarking and ROS 2 result publication. A separate five-class checkpoint from the same project group was measured on a Jetson Orin NX at 38.534 FPS mean model-call throughput, while its ROS 2 result topic reached 29.99 Hz. These group measurements verify the deployment method and hardware path, but they are not presented as measurements of the four-class checkpoint reported here.

Keywords: object detection; YOLO11n; desktop objects; transfer learning; OpenCV; ONNX; Jetson; ROS 2

Experiment at a Glance

Target classes	mouse, keyboard, laptop, and headphones
Dataset	1,343 images and 1,878 annotated objects
Detector	YOLO11n fine-tuned from pretrained weights
Independent test	123 images containing 213 ground-truth objects
Principal result	174 correctly matched objects; recognition rate 81.69%
Completed evidence	checkpoints, test predictions, error cases, ONNX export, a 28.2-second USB-camera recording, and Jetson/ROS 2 core programs
Deployment reference	same-group five-class run: 38.534 FPS model call and 29.99 Hz ROS 2 result topic
Final evidence	rerun the supplied benchmark and node with the current four-class checkpoint on the target Jetson

Contents

1	Introduction	1
1.1	Assignment Requirements	1
2	Technical Background	2
2.1	One-Stage Object Detection	2
2.2	Transfer Learning	3
2.3	YOLO Annotation Representation	3
2.4	Evaluation Metrics	3
3	Dataset	4
3.1	Classes and Sources	4
3.2	Data Splits and Class Distribution	5
4	System Design and Implementation	7
4.1	Processing Pipeline	7
4.2	Model Fine-Tuning	8
4.3	Offline Evaluation and Error Preservation	8
4.4	USB-Camera Inference	9
4.5	ONNX Export	11
5	Experimental Setup and Protocol	11
5.1	Development Environment	11
5.2	Training Configuration	11
5.3	Independent-Test Protocol	12
5.4	Runtime Measurement Boundary	12
6	Results and Analysis	13
6.1	Training Behavior	13
6.2	Independent-Test Metrics	14
6.3	Fixed-Threshold Recognition Rate	14
6.4	Confusion Matrix and Multi-Class Output	15
6.5	Error Patterns	15
6.6	Stored Evidence and Reproducibility	15
7	Jetson Deployment and ROS 2 Integration	17
7.1	ONNX-to-Jetson Path	17
7.2	Same-Group Jetson Reference Run	18
7.3	ROS 2 Result Interface	18
8	Limitations and Improvement Plan	19
9	Conclusion	20
A	Core Run Sequence	22
B	Jetson and ROS 2 Run Commands	22

1 Introduction

Desktop perception is a basic capability in manipulation, service robotics, and human–robot interaction. A robot must distinguish relevant objects and estimate their image locations before a downstream module can associate perception with grasping, navigation, or task planning. A class label alone is insufficient: the system must return both what an object is and where it appears. This requirement makes object detection more suitable than whole-image classification for a workspace containing several objects simultaneously.

The experiment focuses on four common desktop objects: mouse, keyboard, laptop, and headphones. These categories present complementary visual challenges. A mouse is often small relative to the complete image; a keyboard is wide and may appear at strong perspective angles; a laptop changes appearance when open or closed; and headphones vary substantially between over-ear, on-ear, folded, and partially occluded configurations. Illumination, background clutter, distance, and overlap further change the observed appearance.

YOLO treats object localization and classification as a unified prediction task. The original YOLO formulation predicts bounding boxes and associated class probabilities directly from a complete image in one network evaluation [1]. This one-stage design is a natural choice for continuous camera inference. The present experiment adopts YOLO11n, the nano-scale model distributed through the Ultralytics interface, because the assignment requires a practical balance between recognition quality and edge-device speed [4].

The experimental work covers five connected stages. First, a mixed-source four-class dataset is organized in Ultralytics YOLO format. Second, a pretrained YOLO11n checkpoint is fine-tuned and the best validation checkpoint is retained. Third, the model is evaluated on an independent test split using both standard detection metrics and a fixed-threshold matching analysis. Fourth, a USB-camera program displays the class, box, confidence score, and FPS in real time on the development computer. Finally, the selected model is exported to ONNX, and compact programs are supplied for Jetson benchmarking and ROS 2 publication. The report uses a same-group five-class Jetson run to document the shared hardware and method while keeping it separate from measurements of the present four-class checkpoint.

1.1 Assignment Requirements

The course task requires at least two desktop-object classes, student-prepared data and model training, real-time visualization of class labels, boxes, and confidence values, deployment on Jetson, and publication of recognition results through ROS 2. Acceptance additionally requires simultaneous multi-class recognition, a recognition rate of at least 80 percent over 20 or more tested objects, Jetson throughput of at least 5 FPS, and preservation of test results and typical error cases.

Table 1 maps these requirements to the evidence currently available in the project. Shared group measurements are identified separately from current-checkpoint measurements so that the evidence remains auditable.

Table 1. Assignment requirements and current evidence status.

Requirement	Status	Evidence or remaining action
At least two object classes	Completed	Four classes are defined in the dataset configuration.
Train an object detector	Completed	YOLO11n checkpoints and 89-epoch training records are retained.
Display class, box, and confidence	Completed on Windows	The USB-camera program renders all three outputs and FPS.
Simultaneous multi-class output	Completed offline	A test prediction contains all four classes in one image.
At least 80% over 20 objects	Completed on the independent test	174 of 213 objects are matched, giving 81.69%.
Store results and error cases	Completed	123 annotated test predictions and 12 representative error images are retained.
Jetson execution at ≥ 5 FPS	Method verified by group; current rerun required	A same-group five-class run achieved 38.534 FPS on Jetson Orin NX; the supplied benchmark must be rerun with the four-class checkpoint.
ROS 2 result publication	Implemented; current rerun required	The supplied node publishes JSON on <code>/object_detection/result</code> ; the same-group five-class run measured 29.99 Hz.

2 Technical Background

2.1 One-Stage Object Detection

An object detector maps an input image to a set of detections. Each accepted detection contains a class identifier, a confidence score, and a bounding box. In a one-stage detector, feature extraction and dense prediction are optimized as a unified network instead of using a separate region-proposal stage. Figure 1 summarizes the conceptual data path used in this experiment.

Simplified YOLO detection path

Figure 1. Simplified YOLO detection path from the normalized image input to class, box, and confidence outputs.

The backbone extracts hierarchical image features. Early representations respond to edges, texture, and local contrast, whereas deeper representations encode object-level structure. The neck fuses information across spatial scales, which is useful because the dataset includes small mice, elongated keyboards, and larger laptops. The detection head transforms the fused features into class and localization predictions. Candidate predictions are filtered by a confidence threshold, and non-maximum suppression removes redundant highly overlapping boxes.

The selected YOLO11n model contains approximately 182 layers and 2.59 million parameters, with an estimated computational cost of 6.5 GFLOPs at the configured input scale. These values identify a compact detector; they do not directly determine camera FPS because actual speed also depends on the device, runtime, precision, pre-processing, post-processing, and video-display overhead.

2.2 Transfer Learning

The detector is not trained from random initialization. It starts from the official `yolo11n.pt` pretrained checkpoint and is then fine-tuned on the four local classes. Transfer learning reuses general visual representations learned from a large source dataset and adapts them to a smaller task-specific dataset. This approach typically reduces the amount of task data and computation required to reach useful performance. The output checkpoint `best.pt` represents the adapted model selected according to validation performance.

2.3 YOLO Annotation Representation

Ultralytics object-detection labels use one text file per image. Each object is represented by the class identifier and normalized center-format box coordinates [5]:

$$(c, x_c, y_c, w, h), \quad x_c, y_c, w, h \in [0, 1], \quad (1)$$

where c is the zero-based class identifier, (x_c, y_c) is the box center, and (w, h) is its width and height relative to the image dimensions. If pixel coordinates $(x_{\min}, y_{\min}, x_{\max}, y_{\max})$ are available for an image of width W and height H , the normalized representation is

$$x_c = \frac{x_{\min} + x_{\max}}{2W}, \quad y_c = \frac{y_{\min} + y_{\max}}{2H}, \quad w = \frac{x_{\max} - x_{\min}}{W}, \quad h = \frac{y_{\max} - y_{\min}}{H}. \quad (2)$$

2.4 Evaluation Metrics

Let TP , FP , and FN denote true positives, false positives, and false negatives. Precision measures how many retained detections are correct, whereas recall measures how many ground-truth objects are recovered:

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}. \quad (3)$$

The intersection over union (IoU) between a predicted box B_p and a ground-truth box B_g is

$$\text{IoU}(B_p, B_g) = \frac{|B_p \cap B_g|}{|B_p \cup B_g|}. \quad (4)$$

Average precision (AP) integrates the precision–recall relationship for one class. Mean average precision (mAP) averages AP over classes. The metric $\text{mAP}@0.5$ uses IoU 0.5, whereas $\text{mAP}@0.5:0.95$ averages results across IoU thresholds from 0.5 to 0.95 in increments of 0.05,

following the stricter multi-threshold convention popularized by COCO [3]. The numerical confidence shown next to a live detection is different: it is a score for one box, not the global recognition rate or mAP.

3 Dataset

3.1 Classes and Sources

The final task contains four classes in the order shown in Table 2. This order is shared by the dataset configuration, training, evaluation, and camera visualization.

Table 2. Target classes and identifiers.

Identifier	Class name	Desktop interpretation
0	mouse	computer pointing device
1	keyboard	standalone or visibly bounded keyboard
2	laptop	complete laptop computer
3	headphones	wearable over-ear or on-ear audio device

The final dataset combines three provenance groups. Captured or user-provided material anchors the task to local desktop appearance. Controlled scene-expanded images increase variation in object arrangement, scale, and background. Open Images V7 contributes broader real-world appearance and context; Open Images provides large-scale image annotations that include object bounding boxes [2, 7]. These sources are complementary, but scene-expanded samples are not interpreted as independent camera capture events.

Table 3. Dataset composition by provenance group.

Provenance group	Images	Proportion
Captured or user-provided	147	10.95%
Controlled scene-expanded	598	44.53%
Open Images V7	598	44.53%
Total	1,343	100.00%

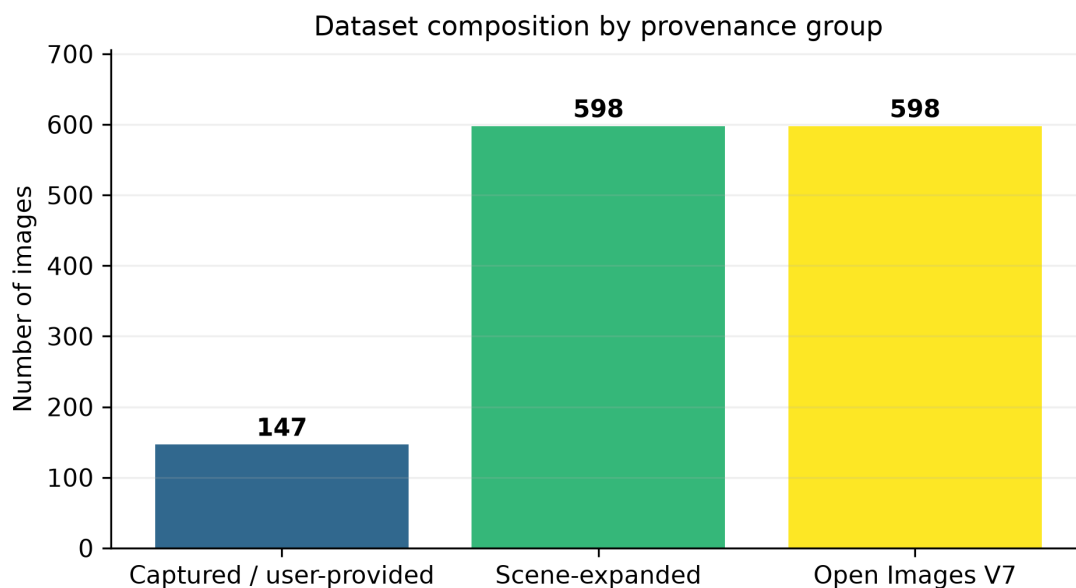


Figure 2. Image count for the three provenance groups in the final dataset.

3.2 Data Splits and Class Distribution

The dataset is divided into training, validation, and test subsets. Training images are used to update network parameters. Validation images are used during training to monitor generalization and select the checkpoint. Test images are reserved for final evaluation. Table 4 reports the exact image and box totals.

Table 4. Dataset split sizes and bounding-box counts.

Split	Images	Image proportion	Bounding boxes
Training	1,047	77.96%	1,400
Validation	173	12.88%	265
Test	123	9.16%	213
Total	1,343	100.00%	1,878

Table 5. Per-class bounding boxes across the three splits.

Class	Training	Validation	Test	Total
mouse	284	61	43	388
keyboard	430	77	66	573
laptop	411	72	61	544
headphones	275	55	43	373
Total	1,400	265	213	1,878

Keyboard has the largest number of boxes and headphones the smallest. The largest-to-smallest ratio is $573/373 \approx 1.54$, which indicates moderate rather than extreme imbalance.

Nevertheless, global mean metrics alone may conceal class-specific weaknesses; the report therefore presents per-class test results.

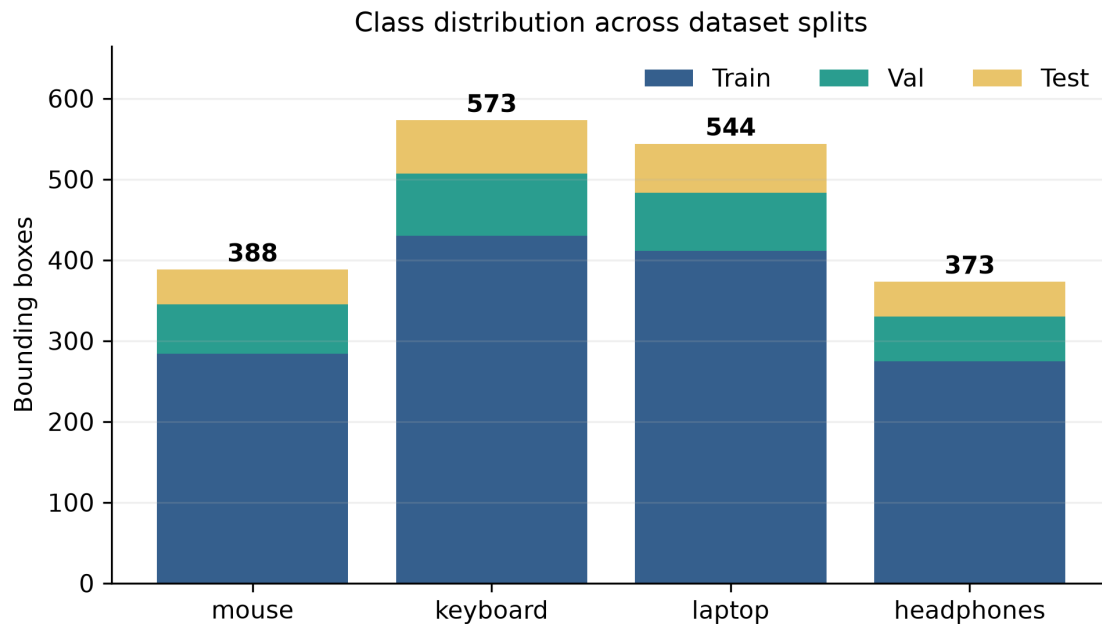


Figure 3. Per-class bounding boxes in the training, validation, and test splits. Values above the bars are class totals.

Figure 4 prioritizes photographs captured or supplied by the project team. The four panels were selected from the organized training, validation, and test directories to show the local devices and desk environments used during development. They are qualitative dataset examples rather than an additional evaluation subset.



Figure 4. Representative team-captured ground-truth samples for (a) mouse, (b) keyboard, (c) laptop, and (d) headphones.

4 System Design and Implementation

4.1 Processing Pipeline

Figure 5 presents the completed processing pipeline. Dataset preparation, training, evaluation, camera inference, and model export have separate entry points so that a failure can be associated with one stage. The final class order and input size are shared by all stages.

Experimental processing pipeline



Figure 5. Experimental pipeline implemented in the current project.

The core implementation is intentionally compact. Table 6 lists the six scripts and the evidence generated by each one.

Table 6. Core implementation modules.

File	Responsibility
prepare_data.py	Assemble the final four-class YOLO dataset and write configuration and summary files.
train.py	Load YOLO11n pretrained weights and fine-tune the detector.
evaluate.py	Evaluate the selected checkpoint on the test split and store annotated predictions.
analyze_errors.py	Match prediction and ground-truth boxes at IoU 0.5, summarize failures, and preserve typical cases.
camera_detect.py	Read the external USB camera and display boxes, classes, confidence scores, and FPS.
export_onnx.py	Export the selected PyTorch checkpoint to an ONNX model.

4.2 Model Fine-Tuning

The training script locates the project root from its own path, initializes YOLO11n from a pretrained checkpoint, and calls the Ultralytics training interface. The essential structure is shown in Listing 1.

Listing 1. Core YOLO11n training call.

```

1 from ultralytics import YOLO
2
3 model = YOLO(project_root / "yolo11n.pt")
4 model.train(
5     data=dataset_root / "data.yaml",
6     epochs=100,
7     imgsz=640,
8     device=0,
9     amp=True,
10    cos_lr=True,
11    close_mosaic=10,
12    seed=20260831,
13    deterministic=True,
14 )

```

The archived training configuration records batch size 16 and four data-loader workers for the run that produced the reported checkpoint. After the experiment, the editable script was set to batch size 8 and two workers to reduce hardware load during optional retraining. Results in this report are attributed to the archived run rather than to the later low-load defaults.

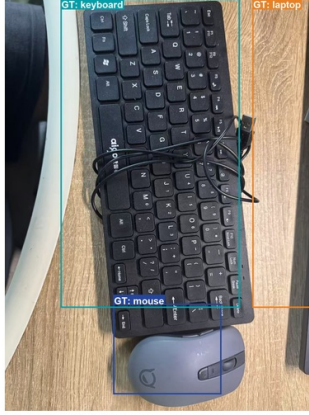
Mosaic augmentation is enabled during most of training and disabled for the last ten epochs. This strategy increases layout and scale variation early in optimization while allowing the model to finish on inputs closer to ordinary image composition. Automatic mixed precision reduces memory and computational cost on the CUDA GPU. A fixed seed and deterministic mode improve repeatability, although complete bitwise reproduction can still depend on the GPU, CUDA, PyTorch, and low-level operator implementations.

4.3 Offline Evaluation and Error Preservation

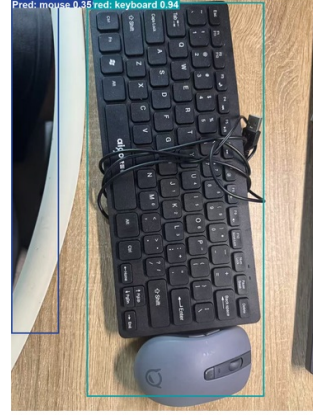
The evaluation script loads `best.pt` and runs the Ultralytics validation interface on the test split. It then predicts all 123 test images at confidence 0.25. The prediction stage saves

annotated images, text labels, and confidence values. Figure 6 compares annotations and saved predictions on two team-captured test photographs.

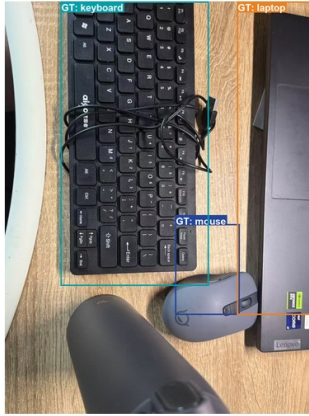
(a) Captured ground truth



Selected-checkpoint prediction



(b) Captured ground truth



Selected-checkpoint prediction

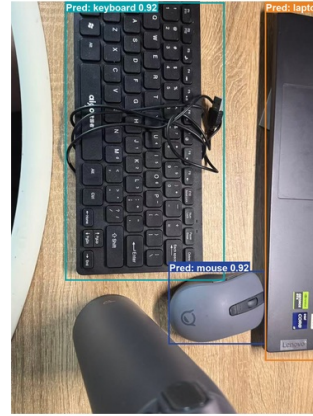


Figure 6. Team-captured independent-test examples, with reference annotations on the left and predictions from the selected checkpoint on the right.

The separate error-analysis script greedily matches each ground-truth box to the unused same-class prediction with the greatest IoU. A match is accepted when IoU is at least 0.5. For each image,

$$FN = N_{GT} - TP, \quad FP = N_{pred} - TP. \quad (5)$$

Images with any false negative or false positive are retained in a CSV table, ranked by an error score that gives missed objects additional weight, and representative cases are copied to a review directory. This supports reproducible failure inspection rather than relying only on successful screenshots.

4.4 USB-Camera Inference

The real-time program uses OpenCV [8] to open camera index 1 through the Windows DirectShow backend. It requests 640 by 480 capture frames, applies YOLO inference at an input size of 640, draws predictions, computes the interval between displayed frames, and overlays FPS. The main inference fragment is shown in Listing 2.

Listing 2. Core real-time camera inference loop.

```

1 camera = cv2.VideoCapture(1, cv2.CAP_DSHOW)
2 camera.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
3 camera.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
4
5 success, frame = camera.read()
6 result = model.predict(
7     frame, imgsz=640, conf=0.25,
8     device="cpu", verbose=False
9 )[0]
10 display = result.plot()

```

The current camera program deliberately uses the CPU for simple Windows-side demonstration. It releases the camera in a `finally` block and closes all OpenCV windows when the user presses Q. This program verifies the inference and visualization path, but its measured speed is not a substitute for a Jetson benchmark.

The supplied recording contains 845 frames at 30 frames per second and lasts approximately 28.2 seconds. Figure 7 presents four chronological frames cropped to the camera content only; the surrounding desktop, editor, and terminal are excluded. The selected frames visibly demonstrate laptop, keyboard, and mouse detections. Their on-screen FPS overlays range from approximately 14.8 to 16.3, but these displayed values remain Windows demonstration evidence rather than a controlled Jetson throughput measurement.



Figure 7. Chronological frames from the provided USB-camera recording. Every panel is cropped to the camera-view region and retains the detector's class, confidence, bounding-box, and FPS overlays.

4.5 ONNX Export

The selected PyTorch model is exported using the Ultralytics export interface [6]. The current configuration uses a fixed 640-pixel input, ONNX opset 17, graph simplification, and a fixed batch dimension:

Listing 3. Export of the selected checkpoint to ONNX.

```
1 YOLO(best_model).export(
2     format="onnx",
3     imgsz=640,
4     opset=17,
5     simplify=True,
6     dynamic=False,
7 )
```

The resulting model has input shape $1 \times 3 \times 640 \times 640$ and output shape $1 \times 8 \times 8400$. ONNX is a framework-neutral representation of a model computation graph and parameters [9]. Export does not retrain the detector. It prepares a portable model that can be imported by an ONNX runtime or converted into a device-specific TensorRT engine.

5 Experimental Setup and Protocol

5.1 Development Environment

Training and offline evaluation were performed on a Windows laptop. Table 7 records the measured hardware and software environment. Storing these versions is important because GPU behavior and export compatibility can vary across software stacks.

Table 7. Training and development-computer environment.

Component	Measured or installed value
CPU	Intel Core i7-13620H, 10 cores and 16 logical processors
System memory	16 GB
GPU	NVIDIA GeForce RTX 4060 Laptop GPU
GPU memory	8,188 MiB reported by the driver
GPU driver	610.88
Python	3.13.14
PyTorch	2.7.0+cu128
CUDA runtime used by PyTorch	12.8; CUDA available
Ultralytics	8.4.136
OpenCV	5.0.0
Camera	USB2.0 HD UVC WebCam, Windows camera index 1

5.2 Training Configuration

Table 8 reports the parameters associated with the archived run. The maximum epoch setting was 100, and the stored results contain 89 complete epochs. The run was stopped after sustained laptop thermal load, while the validation-selected checkpoint had already occurred at epoch 75.

Table 8. Principal YOLO11n fine-tuning parameters for the reported run.

Item	Setting
Task	Four-class horizontal bounding-box detection
Initialization	Official pretrained <code>yolo11n.pt</code>
Maximum / completed epochs	100 configured / 89 complete records
Selected checkpoint	Epoch 75 by validation fitness
Input size	640×640
Batch size / workers	16 / 4 for the archived training run
Training device	CUDA device 0, RTX 4060 Laptop GPU
Automatic mixed precision	Enabled
Optimizer	Ultralytics automatic selection; AdamW recorded in the run summary
Learning-rate schedule	Cosine schedule
Mosaic	Enabled; disabled for the final 10 configured epochs
Randomness control	Seed 20260831; deterministic mode enabled
Outputs	Checkpoints, configuration, epoch metrics, label plot, and training batches

5.3 Independent-Test Protocol

The selected `best.pt` checkpoint is evaluated on 123 test images containing 213 ground-truth boxes. Standard Ultralytics validation generates global and per-class precision, recall, AP, and confusion-matrix results. A second fixed-threshold analysis uses predictions saved at confidence 0.25 and accepts same-class matches at IoU at least 0.5. Each prediction can match no more than one ground-truth box. This second protocol directly counts matched objects and allows comparison with the assignment’s recognition-rate threshold.

The 213-object test exceeds the required minimum of 20 objects. The reported 81.69 percent therefore describes the complete available test split, not a favorable subset selected after viewing the predictions. A separate, manually witnessed 20-object classroom checklist can still be added as acceptance documentation, but it is not needed to compute the current 213-object result.

5.4 Runtime Measurement Boundary

An existing Windows-side CPU timing record reports approximately 65.3 ms for one image prediction, corresponding to about 15.3 predictions per second:

$$\text{FPS}_{\text{model}} \approx \frac{1000}{65.3} = 15.3. \quad (6)$$

This value represents a development-computer model prediction measurement. A displayed camera FPS may also include capture, pre-processing, post-processing, drawing, and window refresh. A Jetson benchmark must specify its own timing boundary, warm-up count, measured-frame count, power mode, model backend, and numerical precision. The supplied `jetson_benchmark.py` fixes the software boundary around one complete `model.predict` call, excludes camera acquisition and visualization, uses 20 warm-up frames followed by 200 measured frames, and writes every measured interval to CSV.

6 Results and Analysis

6.1 Training Behavior

The archived run contains 89 complete epoch records and a total recorded training time of 940.777 s, or approximately 15 min 41 s. Figure 8 reconstructs the training history directly from `results.csv`. The training box and classification losses decrease substantially, while the validation losses stabilize at higher values. Precision, recall, and mAP improve rapidly during the early epochs and then fluctuate within a narrower range.

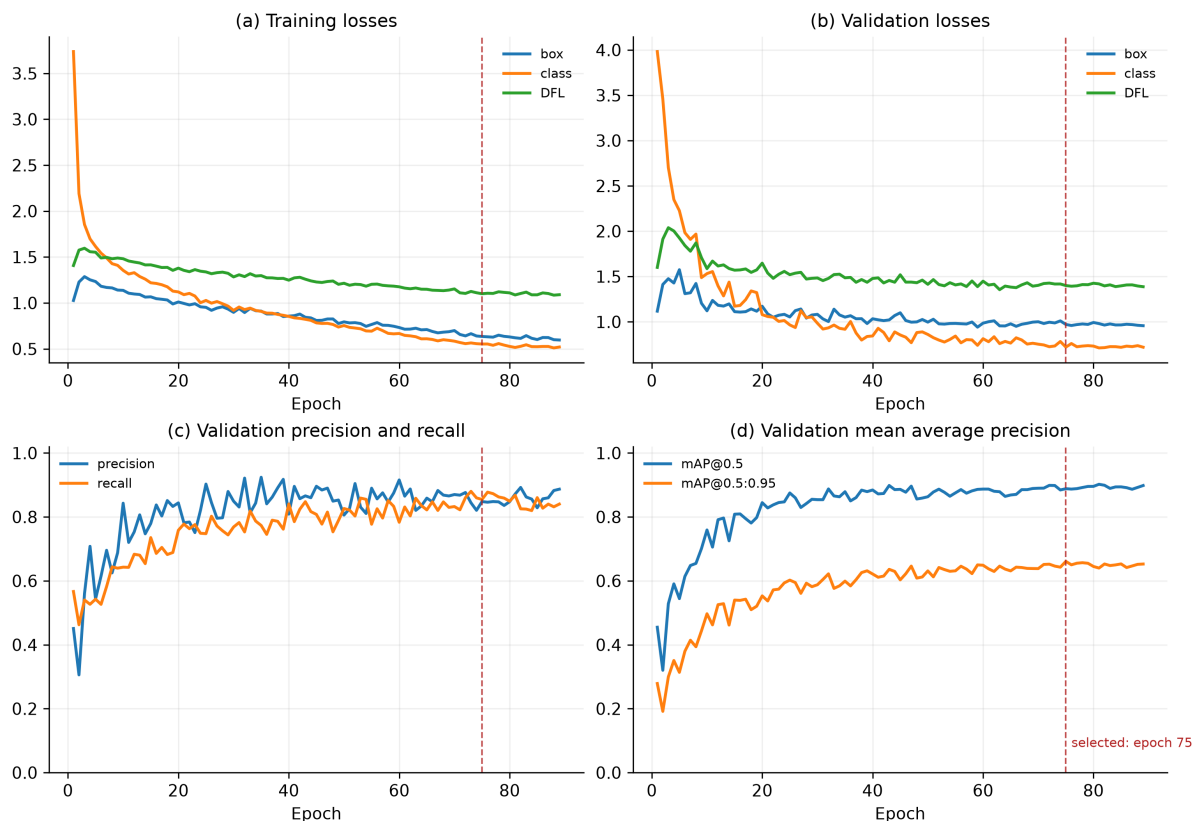


Figure 8. YOLO11n training history. The dashed line marks the validation-selected checkpoint at epoch 75.

The selected checkpoint is associated with the best composite validation fitness rather than the single greatest value of one metric. At epoch 75, precision is 0.84795, recall is 0.85465, mAP@0.5 is 0.88836, and mAP@0.5:0.95 is 0.66088. The highest individual mAP@0.5 in the record is 0.90211 at epoch 81, but its stricter multi-IoU score is lower than at epoch 75. Selecting the composite checkpoint avoids attributing the final model only to the most favorable loose-IoU value.

Table 9. Validation metrics for the selected checkpoint.

Checkpoint	Precision	Recall	mAP@0.5	mAP@0.5:0.95
Epoch 75 / best.pt	0.848	0.855	0.888	0.661

6.2 Independent-Test Metrics

Table 10 gives global and per-class results on the independent test split. Overall precision is 0.882, which is higher than recall 0.762. The retained predictions are therefore comparatively reliable, while missed objects remain the larger limitation. The gap between $\text{mAP}@0.5$ and $\text{mAP}@0.5:0.95$ is $0.810 - 0.564 = 0.246$, showing that localization quality decreases under stricter overlap requirements.

Table 10. Global and per-class detection metrics on the 123-image test split.

Class	Precision	Recall	AP@0.5	AP@0.5:0.95
mouse	0.862	0.744	0.720	0.377
keyboard	0.886	0.818	0.899	0.734
laptop	0.909	0.705	0.803	0.628
headphones	0.870	0.781	0.818	0.518
Overall	0.882	0.762	0.810	0.564

Keyboard has the strongest AP values, including $\text{AP}@0.5:0.95$ of 0.734. Its regular structure and comparatively high number of training boxes may support stable detection. Laptop has the highest precision, 0.909, but recall is only 0.705, indicating that accepted detections are usually correct while a meaningful fraction of annotated laptops remains unmatched. Mouse has the lowest $\text{AP}@0.5:0.95$, 0.377. Small image area, partial visibility, and appearance variation plausibly contribute to this result, although the experiment does not isolate these factors causally. Headphones reach recall 0.781 but remain sensitive to pose and background.

6.3 Fixed-Threshold Recognition Rate

Table 11 summarizes the same-class IoU matching analysis at confidence 0.25 and IoU 0.5. The overall count is

$$\text{Recognition rate} = \frac{174}{213} \times 100\% = 81.69\%. \quad (7)$$

This result exceeds the assignment threshold of 80 percent over at least 20 tested objects. It should not be substituted for mAP: it uses one presentation threshold and a direct match count, whereas AP integrates ranked detections over thresholds.

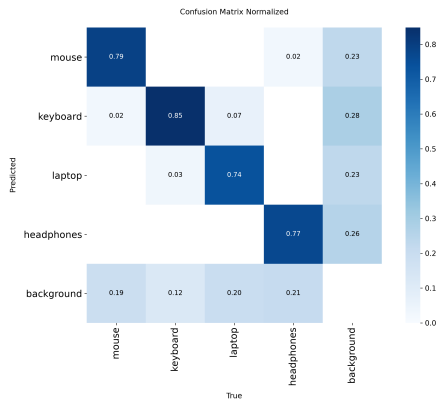
Table 11. Fixed-threshold object matching on the independent test split.

Class	Ground-truth objects	Correctly matched	Recognition rate
mouse	43	32	74.42%
keyboard	66	58	87.88%
laptop	61	50	81.97%
headphones	43	34	79.07%
Overall	213	174	81.69%

6.4 Confusion Matrix and Multi-Class Output

Figure 9a is normalized by true class. Diagonal values are 0.79 for mouse, 0.85 for keyboard, 0.74 for laptop, and 0.77 for headphones. The background row contains 0.19, 0.12, 0.20, and 0.21 for these four classes, respectively, which is consistent with missed detections. Explicit class-to-class confusion is smaller than the background-related errors. Laptop-to-keyboard overlap is plausible in desktop imagery because a visible laptop keyboard shares local structure with a standalone keyboard.

Figure 9b demonstrates simultaneous output for all four classes in one test image. The detector reports laptop 0.89, keyboard 0.33, mouse 0.46, and headphones 0.93. The image proves that the model can produce several classes in a single frame. It is qualitative evidence and does not replace the aggregate test metrics.



(a) Normalized confusion matrix.



(b) Simultaneous four-class output.

Figure 9. Complementary independent-test evidence: quantitative class-confusion structure and a qualitative multi-class prediction. Columns in panel (a) are true classes and rows are predicted classes.

6.5 Error Patterns

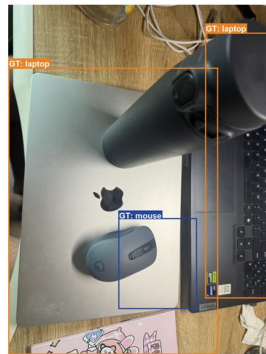
The fixed-threshold comparison identifies 51 test images with at least one false positive or false negative. Twelve representative cases are preserved, and Figure 10 shows four team-captured examples with ground truth on the left and predictions on the right.

The cases reveal several recurring patterns. Complex backgrounds can trigger an incorrect class around a visually salient region. Headphones worn by a person may be small or partly occluded and can be missed entirely. A low-confidence true class may coexist with a false positive of another class. Tight crops and unusual viewpoints also reduce localization agreement even when the semantic category appears visually plausible. These cases motivate additional captured data with small mice, worn or folded headphones, partially occluded laptops, and cluttered desks.

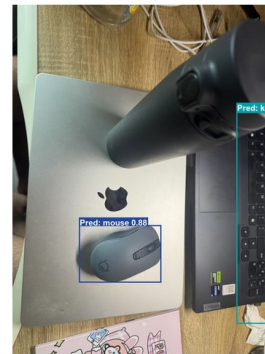
6.6 Stored Evidence and Reproducibility

Table 12 lists the principal artifacts that connect the reported values to files. The project retains 123 annotated prediction images and 12 representative error images. The training CSV, archived arguments, dataset summary, and error summary allow the principal tables in this report to be regenerated.

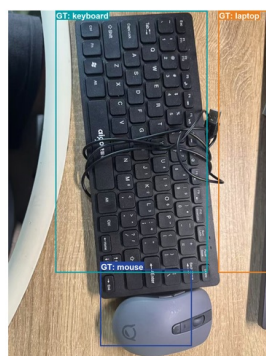
(a) Ground truth



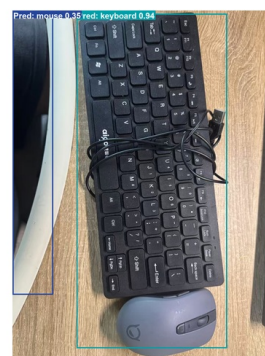
Prediction: FN=3, FP=2



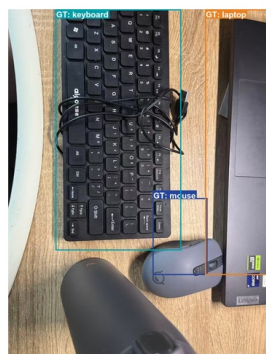
(b) Ground truth



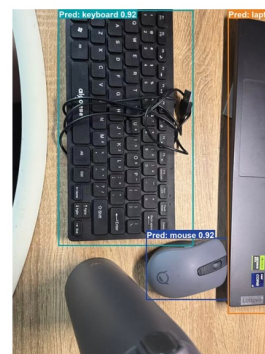
Prediction: FN=2, FP=1



(c) Ground truth



Prediction: FN=1, FP=1



(d) Ground truth



Prediction: FN=1, FP=1

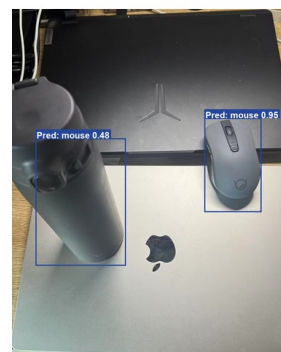


Figure 10. Representative test errors. Each row compares ground truth with the saved prediction; FN and FP denote unmatched reference and predicted boxes.

Table 12. Principal experiment artifacts and their roles.

Artifact	Role
data.yaml	Dataset root, split paths, and class order
SUMMARY.json	Image totals and class-specific box counts
MANIFEST.csv	Per-image split and provenance record
args.yaml	Archived training configuration
results.csv	Per-epoch losses and validation metrics
best.pt / last.pt	Selected and final PyTorch checkpoints
desktop4_test	Curves, confusion matrices, and label/prediction batches
desktop4_test_predictions	123 annotated test outputs and saved prediction labels
ERROR_CASES.csv	Per-image TP, FN, FP, and error ranking
USB-camera MP4	28.2-second Windows demonstration with class, box, confidence, and FPS overlays
best.onnx	Fixed-shape exported deployment model
jetson_benchmark.py	Target-device benchmark with fixed warm-up, measured-frame count, timing boundary, and per-frame CSV output
ros2_detector_node.py	USB-camera detector and JSON publisher on /object_detection/result
ros2_result_subscriber.py	Independent subscriber for readable message verification

7 Jetson Deployment and ROS 2 Integration

7.1 ONNX-to-Jetson Path

The completed export establishes a portable deployment boundary. NVIDIA documents ONNX as a common model-import route for TensorRT, which compiles a trained network into a GPU-specific serialized engine [11, 10]. The supplied programs also accept the original `best.pt` checkpoint, allowing PyTorch FP16 to be tested first and an ONNX or TensorRT backend to be substituted later without changing the measurement logic. Figure 11 summarizes this path.

Deployment path and remaining target-device stages

Figure 11. Deployment path. The checkpoint, ONNX export, Jetson benchmark, and ROS 2 publisher are implemented; the current checkpoint still requires target-device execution.

The target-device evaluation records the exact device model, JetPack, CUDA, TensorRT, power mode, camera device, backend, numerical precision, confidence threshold, and input size. The benchmark opens `/dev/video0`, discards 20 warm-up frames, times 200 complete `model.predict` calls, and stores model-call, preprocessing, inference, and post-processing durations for every frame. Mean and median latency, mean throughput, and the minimum observed rate are printed. This makes the final 5 FPS decision traceable to the Jetson configuration rather than to the RTX 4060 or Windows CPU.

7.2 Same-Group Jetson Reference Run

The group reference report documents a completed five-class deployment on an NVIDIA Jetson Orin NX Engineering Reference Developer Kit Super [13]. Its environment used JetPack 6.2.1, L4T 36.4.4, CUDA 12.6.77, NVIDIA-build PyTorch 2.5.0, ROS 2 Humble, MAXN_SUPER power mode, CUDA device 0, FP16 precision, and a 640-pixel YOLO11n input. The USB camera supplied 640 by 480 frames from `/dev/video0`. TensorRT 10.7.0.23 libraries were installed, but the measured backend was PyTorch FP16 because the active Python environment did not contain the TensorRT binding.

After 20 warm-up frames, that run measured 200 frames. Its mean complete prediction-call time was 25.951 ms, equivalent to 38.534 FPS, and its minimum per-frame call rate was 37.952 FPS. The result topic was independently measured at 29.99 Hz over 15 seconds with a 200-message window. Table 13 makes the evidence boundary explicit.

Table 13. Deployment evidence and attribution boundary.

Evidence	Checkpoint and classes	Measured result	Interpretation
Same-group reference run	Five-class YOLO11n, including cup	25.951 ms mean call; 38.534 FPS mean; 37.952 FPS minimum; 29.99 Hz topic	Verifies the shared Jetson, camera, FP16, and ROS 2 method; exceeds 5 FPS.
Present experiment	Four-class YOLO11n, excluding cup	Benchmark and publisher implemented; target run not available on the development computer	Must be rerun with the present <code>best.pt</code> before attributing Jetson speed to this checkpoint.

The two checkpoints have different class sets, datasets, and learned parameters. Consequently, the reference numbers support feasibility and group-level integration experience, but they are not inserted into the four-class test metrics or relabeled as its runtime. This separation avoids an invalid cross-checkpoint comparison while still preserving work legitimately shared by the group.

7.3 ROS 2 Result Interface

ROS 2 topics implement asynchronous publish/subscribe communication for continuous data streams [12]. The implemented node reads the USB camera, performs one prediction per frame, displays the annotated image, and publishes a compact JSON document in

`std_msgs/String` on `/object_detection/result`. Table 14 defines the implemented payload.

Table 14. Implemented structured fields for ROS 2 detection publication.

Field	Type	Meaning
timestamp	floating point	node-clock timestamp
frame_id	integer	processed frame counter
fps	floating point	defined runtime rate
detections	list	all accepted detections in the frame
class_id	integer	identifier from 0 through 3
class_name	string	mouse, keyboard, laptop, or headphones
confidence	floating point	score for the current box
bbox	array	pixel coordinates (x_1, y_1, x_2, y_2)

A frame with no accepted detection should still publish its timestamp and frame identifier with an empty detection list. This lets a subscriber distinguish normal operation with no current target from a failed publisher. The implementation follows this rule and uses a queue depth of 10. Publication frequency is measured independently with `ros2 topic hz`; it is not necessarily identical to neural-network inference FPS because serialization, drawing, camera acquisition, display refresh, and queue behavior add work.

Listing 4. Core ROS 2 result publication implemented by the detector node.

```

1 message.data = json.dumps({
2     "timestamp": self.get_clock().now().nanoseconds / 1e9,
3     "frame_id": self.frame_id,
4     "fps": round(self.smoothed_fps, 2),
5     "detections": detections,
6 })
7 self.publisher.publish(message)

```

The independent subscriber parses this JSON and prints the frame identifier, smoothed node FPS, class name, and confidence of every detection. A second terminal command, `ros2 topic echo /object_detection/result --once`, preserves an example payload, and `ros2 topic hz /object_detection/result --window 200` measures delivered message frequency. Thus implementation is complete; only execution of the present checkpoint on the physical Jetson remains as replacement evidence.

8 Limitations and Improvement Plan

The current experiment satisfies the four-class training and offline recognition-rate portions, but several limitations remain. First, the dataset contains only 147 captured or user-provided images; the remainder consists of public images and controlled scene expansion. This improves coverage but cannot fully reproduce the local camera, desk material, room lighting, and user interaction. Additional independent captures should therefore be prioritized over repeated variants of existing scenes.

Second, class performance is uneven. Mouse has the lowest strict localization AP, and laptop has the lowest recall. More small-object mouse images and occluded laptop examples

should be collected. Headphones require examples of worn, folded, side-view, and partial configurations. Annotations should be reviewed consistently, particularly when a laptop keyboard and a standalone keyboard can both be visible.

Third, the confidence threshold trades false positives against missed detections. The current 0.25 threshold supports quantitative recall-oriented evaluation but can display low-confidence boxes during demonstration. A classroom visualization may use a threshold between 0.35 and 0.50 after checking that simultaneous recognition remains stable. The chosen display threshold must be recorded and should not silently replace the threshold used for the reported test metrics.

Fourth, the Windows camera program uses the CPU, while the Jetson programs default to CUDA device 0 and FP16. The exported ONNX model or a TensorRT engine can be tested after the PyTorch checkpoint, but any backend conversion must be followed by an accuracy comparison against `best.pt`. The same-group measurements show that the shared PyTorch FP16 path comfortably exceeded 5 FPS; they do not guarantee identical speed for the present checkpoint.

Finally, future reporting should preserve distinct measurement boundaries. Dataset AP, fixed-threshold match rate, model-call FPS, complete camera-loop FPS, and ROS 2 topic frequency answer different questions. Keeping them separate prevents a successful screenshot or a fast isolated inference call from being misreported as end-to-end system performance.

9 Conclusion

This experiment establishes a complete development-computer pipeline for four-class desktop object detection. A 1,343-image dataset containing 1,878 boxes is organized for mouse, keyboard, laptop, and headphones. YOLO11n transfer learning produces a selected checkpoint with validation $\text{mAP}@0.5$ of 0.888 and $\text{mAP}@0.5:0.95$ of 0.661. On 123 independent test images, the detector reaches precision 0.882, recall 0.762, $\text{mAP}@0.5$ 0.810, and $\text{mAP}@0.5:0.95$ 0.564. Fixed-threshold matching identifies 174 correct detections among 213 annotated objects, for an overall recognition rate of 81.69 percent, exceeding the assignment's 80 percent threshold.

The implementation preserves all annotated test predictions and representative failures, demonstrates simultaneous multi-class output, and provides a USB-camera program that displays classes, boxes, confidence scores, and FPS. The supplied 28.2-second recording preserves this camera demonstration, and the report includes camera-only cropped frames. The best checkpoint has been exported to an ONNX model with fixed $1 \times 3 \times 640 \times 640$ input. A reproducible Jetson benchmark, a ROS 2 JSON publisher, and a verification subscriber have been implemented as compact core programs. A same-group five-class run on Jetson Orin NX achieved 38.534 FPS mean model-call throughput and a 29.99 Hz result topic, verifying the shared deployment path. Because that run used a different checkpoint, the supplied programs must be executed once more with the present four-class `best.pt` before those target-device values can be attributed to this experiment.

References

- [1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 779–788. Available: https://openaccess.thecvf.com/content_cvpr_2016/html/Redmon_You_Only_Look_CVPR_2016_paper.html
- [2] A. Kuznetsova, H. Rom, N. Alldrin, J. Uijlings, I. Krasin, J. Pont-Tuset, S. Kamali, S. Popov, M. Mallocci, A. Kolesnikov, T. Duerig, and V. Ferrari, “The Open Images Dataset V4: Unified Image Classification, Object Detection, and Visual Relationship Detection at Scale,” *International Journal of Computer Vision*, vol. 128, pp. 1956–1981, 2020.
- [3] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, “Microsoft COCO: Common Objects in Context,” in *Computer Vision – ECCV 2014*, 2014, pp. 740–755.
- [4] Ultralytics, “Ultralytics YOLO Documentation,” accessed September 2026. Available: <https://docs.ultralytics.com/>
- [5] Ultralytics, “Object Detection Datasets Overview,” accessed September 2026. Available: <https://docs.ultralytics.com/datasets/detect/>
- [6] Ultralytics, “Model Export with Ultralytics YOLO,” accessed September 2026. Available: <https://docs.ultralytics.com/modes/export/>
- [7] Google, “Open Images Dataset V7 and Extensions,” 2022, accessed September 2026. Available: <https://storage.googleapis.com/openimages/web/index.html>
- [8] OpenCV, “OpenCV Documentation,” accessed September 2026. Available: <https://docs.opencv.org/>
- [9] ONNX, “Introduction to ONNX,” accessed September 2026. Available: <https://onnx.ai/onnx/intro/>
- [10] NVIDIA Corporation, “NVIDIA TensorRT Product Family,” accessed September 2026. Available: <https://docs.nvidia.com/tensorrt/>
- [11] NVIDIA Corporation, “How TensorRT Works,” accessed September 2026. Available: <https://docs.nvidia.com/deeplearning/tensorrt/latest/architecture/how-trt-works.html>
- [12] Open Robotics, “ROS 2 Humble Documentation: Interfaces (Topics, Services, Actions),” accessed September 2026. Available: <https://docs.ros.org/en/humble/Concepts/Basic/Interfaces-Topics-Services-Actions.html>
- [13] S. Li, “Desktop Object Detection and ROS 2 Result Publishing with YOLO11n,” Robotics Integration Group Project Laboratory Report, Ocean University of China, September 2026. Internal group report.

A Core Run Sequence

The logical execution sequence for rebuilding the completed artifacts is:

1. run `prepare_data.py` to create the four-class dataset;
2. run `train.py` to fine-tune YOLO11n;
3. run `evaluate.py` to evaluate the test split and save predictions;
4. run `analyze_errors.py` to create the match summary and error review set;
5. run `export_onnx.py` to create `best.onnx`;
6. run `camera_detect.py` separately for Windows USB-camera demonstration;
7. run `jetson_benchmark.py` on the Jetson for the fixed 20-frame warm-up and 200-frame benchmark;
8. run `ros2_detector_node.py`, then verify it with `ros2_result_subscriber.py` and `ros2 topic hz`.

The Windows command below forces the project virtual environment and avoids accidentally using a system Python installation without Ultralytics:

Listing 5. Logical camera run command; replace PROJECT ROOT with the local workspace path.

```
1 & "PROJECT_ROOT/.venv_dataset/Scripts/python.exe" \
2   "PROJECT_ROOT/training code/camera_detect.py"
```

B Jetson and ROS 2 Run Commands

After the three programs and `best.pt` are copied to `/desktop_detector` on the Jetson, the four-class benchmark is executed as follows:

Listing 6. Jetson benchmark command.

```
1 cd ~/desktop_detector
2 python3 jetson_benchmark.py --model models/best.pt \
3   --source /dev/video0 --device 0 --half \
4   --warmup 20 --frames 200
```

The ROS 2 publisher and its independent checks are started in separate terminals:

Listing 7. ROS 2 publisher and verification commands.

```
1 source /opt/ros/humble/setup.bash
2 python3 ros2_detector_node.py --ros-args \
3   -p model:=models/best.pt -p source:=/dev/video0 \
4   -p device:=0 -p half:=true -p display:=true
5
6 ros2 topic echo /object_detection/result --once
7 ros2 topic hz /object_detection/result --window 200
8 python3 ros2_result_subscriber.py
```

The benchmark writes `outputs/benchmark/jetson_camera.csv`. When the physical Jetson is available, the clearly labeled same-group deployment reference can be replaced with the current checkpoint's console output, CSV, and one ROS 2 topic screenshot.